AMAYASOFT

Manual técnico y funcional del producto

Edición: Producto final Fecha de actualización: 3 de noviembre de 2026 Tecnología principal: ASP.NET Core MVC sobre .NET 9 Base de datos: Microsoft SQL Server

> *Este manual describe la organización y las funciones implementadas en el producto. No incluye registros de clientes, ventas, compras ni productos, capturas con información personal, contraseñas, tokens, cadenas de conexión ni resultados comerciales inventados. Los nombres de entidades y campos que aparecen son parte de la estructura técnica de AmayaSoft.*

Índice

1. Resumen del sistema
2. Objetivo y alcance
3. Tecnologías y estructura
4. Arquitectura de la aplicación
5. Usuarios, roles y permisos
6. Módulos funcionales
7. Flujos operativos principales
8. Modelo de datos
9. Integración de facturación electrónica
10. Seguridad, consistencia y errores
11. Instalación, configuración y despliegue
12. Pruebas y verificación
13. Limitaciones y estado conocido
14. Guía breve para la exposición
15. Glosario
16. Mapa de archivos relevantes
17. Referencias

1. Resumen del sistema

AmayaSoft es una aplicación web para apoyar la operación de un minimarket. Centraliza el catálogo de productos, el registro de clientes y proveedores, las ventas de punto de venta, la caja por turno, las compras, los gastos, los movimientos de inventario y distintos reportes. Incluye autenticación por usuario y roles, además de una integración configurable con Factiliza para consultar documentos y emitir comprobantes electrónicos.

La aplicación sigue el patrón Model–View–Controller (MVC) de ASP.NET Core. Los controladores reciben las solicitudes web, los servicios agrupan reglas de negocio, Entity Framework Core consulta y actualiza SQL Server, y las vistas Razor generan la interfaz HTML.

La venta y la emisión fiscal son procesos separados. Registrar una venta guarda la operación interna y actualiza el inventario; la emisión a Factiliza se inicia posteriormente mediante una acción exclusiva del administrador. La aceptación fiscal se determina por la respuesta de SUNAT y su CDR, no por la mera recepción HTTP de Factiliza.

2. Objetivo y alcance

Objetivo

Proporcionar una herramienta integrada para registrar y consultar operaciones comerciales de un minimarket, mantener coherentes ventas, caja e inventario, y habilitar la emisión electrónica controlada de boletas y facturas.

Alcance implementado

- Acceso con ASP.NET Core Identity y separación de funciones mediante roles.
- Mantenimiento de productos, categorías, clientes y proveedores.
- Punto de venta con validación de precios, cantidades, descuentos y medios de pago.
- Apertura, seguimiento y cierre de caja por usuario.
- Registro de compras y actualización de stock y costo.
- Kardex con historial y ajustes de inventario.
- Gastos vinculados al turno de caja.
- Dashboard, reportes de ventas y estimación sencilla de reposición.
- Asistente de consultas basado en interpretación de frases y consultas SQL predefinidas.
- Integración HTTP con Factiliza para consulta DNI/RUC y emisión/consulta de documentos fiscales.
- Impresión desde el navegador de tickets internos y etiquetas.

Fuera del alcance actual

- Envío de comprobantes al correo del cliente.
- Sincronización automática de estados fiscales posteriores al primer envío.
- Módulos fiscales de notas de crédito/débito y todas las modalidades tributarias.
- Pronóstico estadístico avanzado, aprendizaje automático o conexión a un modelo de lenguaje externo para el asistente.
- Gestión de pagos o devoluciones bancarias integradas.

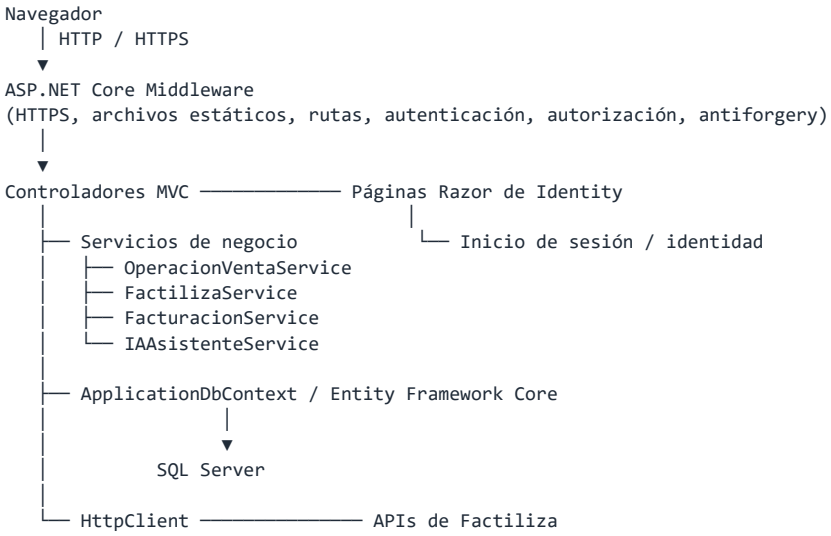
3. Tecnologías y estructura

Área	Tecnología o mecanismo	Uso en el sistema
Plataforma	.NET 9 / ASP.NET Core	Hospedaje web, controladores, middleware y Razor Pages de identidad
Interfaz	ASP.NET Core MVC, Razor, HTML, CSS y JavaScript	Formularios, listados, punto de venta y páginas operativas
Persistencia	Entity Framework Core 9	Mapeo de entidades y consultas a SQL Server
Motor de datos	Microsoft SQL Server	Usuarios, catálogos y datos operativos
Identidad	ASP.NET Core Identity	Autenticación, contraseñas, roles, cookies y bloqueo por intentos fallidos
API externa	HttpClient tipado	Consulta documental y facturación con Factiliza
Gráficos	ECharts	Gráficos del dashboard
Pruebas	Consola .NET de integración	Pruebas de reglas de negocio con base de pruebas aislada y respuestas HTTP simuladas

Organización del código

- `Controllers/`: endpoints MVC y coordinación entre vistas, servicios y persistencia.
- `Services/`: reglas de venta, payload fiscal, consulta Factiliza, facturación, asistente y utilidades.
- `Models/`: entidades persistidas y modelos de presentación.
- `Data/`: `ApplicationDbContext`, actualización aditiva del esquema, corrección horaria y soporte de datos de demostración.
- `Views/`: páginas Razor organizadas por módulo.
- `Areas/Identity/`: páginas de inicio de sesión y configuración de autenticación de dos factores.
- `Migrations/`: migración inicial y snapshot de Entity Framework Core.
- `Tests/`: suite de integración y herramienta manual de QA separada de la batería automática.
- `wwwroot/`: CSS, JavaScript, imágenes y recursos estáticos.

4. Arquitectura de la aplicación



Ciclo de una solicitud

1. El navegador solicita una página o envía un formulario.
2. ASP.NET Core aplica la autenticación, autorización y validación antiforgery configuradas.
3. El controlador valida el contexto de la solicitud y llama a servicios o consultas EF Core.
4. Las operaciones que afectan caja, ventas o inventario pueden abrir una transacción SQL y adquirir un bloqueo de aplicación para coordinar operaciones concurrentes.
5. El resultado se persiste en SQL Server y se devuelve como vista Razor, JSON, redirección o archivo.
6. Las solicitudes externas a Factiliza se realizan desde un `HttpClient` tipado. No se siguen redirecciones HTTP para evitar reenviar payloads a otro host.

Separación de responsabilidades

- Vista: captura datos y muestra información; no es autoridad para precio, stock ni permisos.
- Controlador: aplica autorización de endpoint, antiforgery, propiedad de recursos y flujo de navegación.
- Servicio: recalcula valores sensibles, coordina transacciones y comunica con servicios externos.

- Base de datos: conserva operaciones, relaciones, estados y restricciones únicas.

5. Usuarios, roles y permisos

El sistema utiliza cuentas de Identity y roles. El proyecto contempla principalmente Administrador y Cajero.

Área	Administrador	Cajero
Inicio y consultas operativas	Acceso autenticado	Acceso autenticado
Productos, categorías y proveedores	Gestión	Sin gestión administrativa
Clientes	Gestión completa	Consulta/alta según las acciones habilitadas
Ventas	Historial global	Sus propias ventas
Registrar venta	Sí	Sí, con descuentos sujetos a la regla del servicio
Abrir/cerrar caja	Su turno	Su turno
Historial de caja	Consulta global	Consulta de sus turnos
Compras	Sí	No
Ajustes manuales de Kardex	Sí	Consulta de movimientos
Reporte de ventas y predicción	Sí	No
Emitir comprobante electrónico	Sí, manualmente	No
Anular fiscalmente	Sí, manualmente	No
Consultas del asistente	Consultas globales disponibles al rol	Consultas limitadas a sus operaciones y datos permitidos

Los permisos se aplican en el servidor con atributos de autorización y comprobaciones sobre el propietario de la venta. Ocultar un botón en una vista no reemplaza la autorización del endpoint.

La política configurada para nuevas contraseñas solicita longitud mínima de 12 caracteres, un dígito y un carácter no alfanumérico. Identity bloquea temporalmente la cuenta tras cinco intentos fallidos consecutivos durante el periodo configurado. Las operaciones de emisión y anulación en Producción solicitan además volver a validar la contraseña vigente del administrador.

6. Módulos funcionales

6.1 Inicio de sesión y usuarios

Identity administra el inicio de sesión, las cookies, los roles y la persistencia de usuarios. El administrador gestiona cuentas, perfil básico y asignación de roles. El perfil complementario almacena DNI, nombres y apellidos.

Verificación en dos pasos con Authy

AmayaSoft incorpora configuración de verificación en dos pasos basada en códigos temporales de autenticador de ASP.NET Core Identity (TOTP). La interfaz guía al usuario para vincular su cuenta con Authy escaneando un QR o ingresando la clave manual, confirmar el código de seis dígitos y guardar códigos de recuperación. Cuando el segundo factor queda activado para una cuenta, el inicio de sesión solicita un código Authy; también se puede desactivar confirmando un código vigente. La activación es por cuenta y no está forzada globalmente para todos los usuarios. Authy actúa como aplicación autenticadora: AmayaSoft no llama a una API de Twilio/Authy.

6.2 Dashboard

El dashboard calcula datos desde las tablas operativas y muestra indicadores y gráficos. Incluye ventas activas del día, conteos de catálogo, una valorización estimada de stock a precio, serie de ventas de siete días, hasta seis productos más vendidos, alertas por stock mínimo, comparación de meses y ventas agrupadas por usuario.

Los montos presentados son agregaciones del estado actual de la base de datos; el manual no incorpora resultados concretos de registros.

6.3 Productos y categorías

El catálogo mantiene nombre, descripción, código de barras, imagen, precio, costo, stock, stock mínimo, estado y categoría. El producto puede desactivarse sin borrar necesariamente el historial asociado. El catálogo de categorías organiza los productos y también permite manejar estados activos/inactivos.

Las ediciones de producto no deben utilizarse para corregir stock histórico: los cambios de stock se registran desde compras o Kardex.

6.4 Clientes y proveedores

El registro de cliente contempla documento, nombre, teléfono, correo, dirección y estado. El formulario puede consultar DNI/RUC en Factiliza para proponer información que el usuario revisa y guarda.

El proveedor mantiene RUC, razón social, contacto, teléfono, correo, dirección y estado. La consulta de RUC es una integración de consulta documental; no equivale a una validación contractual o tributaria completa.

El correo del cliente se conserva como dato en la ficha, pero no se incluye en el payload fiscal de Factiliza ni se utiliza para enviar automáticamente el comprobante.

6.5 Punto de venta

El punto de venta requiere una caja abierta para el usuario. Permite buscar productos y clientes, seleccionar cantidades y método de pago, aplicar descuento y registrar la operación. La autoridad de los precios y subtotales es el servidor.

Al confirmar, el servicio recalcula el total con datos de SQL, valida producto activo, existencia de stock, cliente, descuento y forma de pago. Los métodos contemplados incluyen efectivo, tarjeta, Yape, Plin y QR. La regla del servicio aplica un límite de descuento para cajeros y un recargo configurado en código para pago con tarjeta; la factura y boleta se generan posteriormente, no en el paso de cobro.

El sistema conserva un identificador idempotente por operación. Un reintento idéntico puede recuperar la venta ya creada; reutilizar el mismo identificador con contenido diferente se rechaza.

6.6 Caja

La caja se administra por usuario y turno. El flujo incluye apertura con monto inicial, cálculo de ventas activas del turno, gastos asociados, monto efectivo y cierre con importe contado. El historial limita al cajero a sus turnos y permite al administrador ver el historial global.

Una anulación posterior al cierre genera un ajuste asociado, pero no modifica retroactivamente el monto histórico del cierre ni inicia una devolución bancaria.

6.7 Compras

El módulo de compras está restringido al administrador. Valida proveedor activo, número de documento, productos, cantidades, costos y total. La compra se registra dentro de una transacción; sus detalles aumentan stock, actualizan el costo actual del producto y registran movimientos de entrada en inventario.

6.8 Gastos

Un usuario autenticado puede registrar un gasto operativo si tiene una caja abierta. El registro guarda concepto, monto, usuario, fecha y vínculo con el turno. El historial de gastos es administrativo. Las operaciones de gasto utilizan el mismo bloqueo de operaciones comerciales.

6.9 Kardex e inventario

El Kardex permite consultar movimientos por producto y rango de fechas. Las compras generan entradas, las ventas generan salidas y las correcciones manuales registran ajustes con usuario, fecha y motivo. El ajuste manual de inventario es exclusivo del administrador.

6.10 Etiquetas y ticket

El ticket interno resume la operación comercial para impresión desde el navegador. Se distingue del documento fiscal electrónico. El módulo de etiquetas construye una plantilla imprimible para los productos seleccionados; no envía datos a Factiliza.

6.11 Reportes

El reporte de ventas está restringido al administrador y permite consultar un periodo. Muestra fecha, cliente, cajero, forma de pago, artículos, total y costo histórico cuando esté disponible. Las ventas antiguas sin costo histórico se presentan sin una estimación inventada de ese costo.

6.12 Estimación de reposición

La pantalla de predicción está restringida al administrador. El cálculo actual es una heurística simple: toma unidades vendidas en una ventana de 30 días, obtiene un promedio diario, proyecta siete días y compara demanda proyectada más stock mínimo con el stock actual. El resultado orienta la revisión; no es una predicción estadística ni considera automáticamente plazos del proveedor, estacionalidad o promociones.

6.13 Asistente de consultas

El asistente interpreta frases mediante patrones y filtros de fechas, y llama consultas EF Core/SQL predefinidas. No ejecuta SQL generado por el usuario ni utiliza un modelo LLM externo para tomar acciones. Las operaciones de escritura se rechazan y deben ejecutarse desde sus módulos. El acceso a ciertas consultas depende del rol y las ventas del cajero se limitan a las propias.

Puede resumir ventas, métodos de pago, compras, gastos, stock, clientes frecuentes, movimientos de productos, caja y estados de comprobantes en el alcance implementado. El campo de pregunta limita la longitud y la petición tiene timeout.

6.14 Herramientas de demostración y datos semilla

El código incluye una rutina de inicialización y un controlador administrativo para generar datos de demostración. El endpoint `SeedData/Generar` está limitado al entorno `Development` y requiere una solicitud POST de administrador con antiforgery. La rutina automática de inicialización se ejecuta en `Development` o cuando se activa explícitamente `StartupDatabaseTasks:Enabled` fuera de Production. Si necesita crear el administrador inicial, exige la variable secreta `SeedData__InitialAdminPassword`; ya no hay una contraseña predeterminada en el código. Este manual no ejecuta ni reproduce las filas que el sembrador pudiera generar.

7. Flujos operativos principales

7.1 Registrar una venta

```
Abrir caja
  → seleccionar productos y cantidades
  → seleccionar cliente y medio de pago
  → servidor valida solicitud y obtiene precios de SQL
  → servidor recalcula descuento, recargo y total
  → transacción guarda venta, detalles, stock y movimientos
  → venta queda Pendiente de emisión fiscal
  → administrador revisa el detalle y decide si emite el comprobante
```

Si falla la validación antes de confirmar, no debe registrarse parcialmente la operación. La venta no depende de que Factiliza esté disponible para confirmar el cobro en AmayaSoft.

7.2 Cálculo y consistencia del cobro

El navegador envía una solicitud que contiene identificador de operación, productos, cliente, descuento, método e importe esperado. El backend no confía en los precios ni subtotales enviados por el cliente: vuelve a leer productos activos, precios y costos de SQL, verifica stock, agrupa líneas repetidas por producto y calcula los importes.

La transacción guarda la cabecera, detalles y movimientos de stock. Un bloqueo SQL coordina operaciones concurrentes y el producto se lee con bloqueo de fila durante la venta. Esto reduce el riesgo de vender dos veces la última unidad.

7.3 Anular una venta en AmayaSoft

La anulación de la venta en AmayaSoft es administrativa, requiere motivo y utiliza antiforgery. Actualiza estado, fecha, responsable y motivo, y repone el stock una sola vez. Si la venta tiene comprobante fiscal aceptado, la baja fiscal es un paso externo independiente que debe realizar el administrador; anular el registro en AmayaSoft no demuestra que SUNAT haya recibido una nota de baja.

7.4 Registrar una compra

El administrador selecciona proveedor y productos, ingresa número documental, cantidades y costos. El servicio calcula subtotales y total, impide duplicar número de documento para un mismo proveedor, guarda la compra en transacción, aumenta stock y registra el movimiento en Kardex.

7.5 Abrir y cerrar caja

La apertura registra el usuario, fecha y monto inicial. Durante el turno se agregan las ventas activas y los gastos vinculados. El cierre calcula ventas, efectivo y gastos desde la base de datos, registra el importe contado y marca el turno como cerrado.

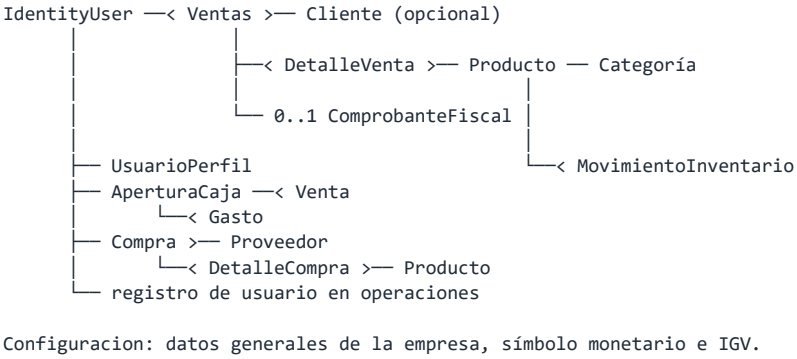
7.6 Emitir un comprobante fiscal

```
Venta activa en estado Pendiente
  → administrador abre el detalle fiscal
  → revisa cliente, documento, importe, IGV y tipo previsto
  → confirma el envío en el ambiente indicado
  → en Producción vuelve a introducir su contraseña
  → el servicio reserva serie y correlativo en SQL
  → persiste payload fiscal y estado previo al envío
  → envía a Factiliza
  → guarda respuesta, hash, mensaje, HTTP y CDR si existen
```

No se permite reenviar ciegamente un documento que tenga intento, rechazo o resultado incierto. Si la conexión se interrumpe o el proveedor presenta estados contradictorios, debe reconciliarse en Factiliza antes de cualquier nueva acción.

8. Modelo de datos

El modelo se documenta por entidades y relaciones; este esquema no contiene registros de negocio.



Entidades principales

Entidad	Responsabilidad y campos de dominio
`Producto`	Catálogo, código, precio, costo actual, stock, mínimo y categoría
`Categoria`	Organización y estado de categorías
`Cliente`	Documento, nombre, teléfono, correo, dirección y estado
`Proveedor`	RUC, razón social y contacto
`Venta`	Usuario, caja, fecha, total, descuento, método, estado e idempotencia
`DetalleVenta`	Producto, cantidad, precio, subtotal y costo histórico opcional
`Compra`	Proveedor, usuario, fecha y documento de proveedor
`DetalleCompra`	Producto, cantidad, costo unitario y total
`AperturaCaja`	Turno, montos inicial/cierre, ventas, efectivo, gastos y ajustes
`MovimientoInventario`	Producto, tipo, cantidad, fecha, usuario, referencia y motivo
`Gasto`	Concepto, monto, usuario, caja y fecha
`ComprobanteFiscal`	Ambiente, tipo, serie, correlativo, estado, respuesta y CDR
`Configuracion`	Datos generales y fiscales de la empresa, y porcentaje de IGV
`UsuarioPerfil`	Datos complementarios de personal vinculados a Identity

Restricciones importantes

- `OperacionId` de venta tiene índice único filtrado cuando no es nulo.
- El número fiscal tiene índice único por ambiente, RUC, serie y correlativo.
- `ComprobanteFiscal` tiene relación uno a uno con la venta y restringe eliminación en cascada.
- El DNI de perfil de usuario tiene índice único.
- Las ventas conservan el costo histórico del detalle cuando se registran; los reportes no sustituyen ese valor por el costo actual del producto.

9. Integración de facturación electrónica

9.1 Integraciones Factiliza

Hay dos usos diferentes de Factiliza:

1. Consulta DNI/RUC: servicio de consulta documental con su propio token y base URL.
2. Facturación SUNAT: emisión de boleta/factura, consulta de PDF/XML y solicitudes de anulación con token y ambientes QA/Producción separados.

La URL base de facturación está fijada en código por ambiente; la credencial no forma parte del repositorio y debe llegar como variable de entorno. El token de consulta y el token de facturación no son intercambiables.

9.2 Selección de tipo de documento y payload

- El tipo de documento fiscal es factura cuando el documento del cliente tiene 11 dígitos; de lo contrario se arma boleta.
- La factura exige RUC y dirección del cliente.
- El payload incluye emisor, cliente, serie, correlativo, moneda, fecha de Perú, detalle, impuestos, medio de pago y leyenda del total en letras.
- El porcentaje de IGV se toma de Configuraciones, con valor predeterminado del código cuando no existe registro de configuración.
- El correlativo se envía a Factiliza como número en texto sin padding; los ceros a la izquierda corresponden a presentación visual.
- El correo del cliente no se incorpora al payload de emisión.

9.3 Configuración por ambiente

La facturación solo queda habilitada cuando la variable general, el ambiente correcto, el token propio del ambiente, el RUC, ambas series y sus correlativos iniciales son válidos. Producción requiere, además, que `SeriesConfirmadas` esté activo y que el proceso se ejecute bajo el entorno `Production`.

El archivo `appsettings.Production.json` mantiene apagados por defecto el envío y la confirmación de series. La URL y las claves secretas nunca se obtienen de contenido suministrado por el navegador.

9.4 Criterio de aceptación y archivos

Una respuesta exitosa de HTTP por sí sola no acredita el comprobante. El servicio requiere respuesta HTTP exitosa, éxito del proveedor, éxito SUNAT y CDR con código `0` para marcar el estado de aceptación. El CDR se conserva si se devuelve. PDF y XML son consultas posteriores; obtener un error HTTP 200 o una vista de detalle en el portal no es equivalente a una aceptación con CDR.

9.5 Estado operativo conocido

Durante una prueba manual comunicada por el usuario, Factiliza reportó un error al firmar con OpenSSL porque no pudo convertir el parámetro recibido en una clave privada. El portal mostró el documento pendiente de envío y no había CDR de aceptación. En la respuesta observada, AmayaSoft clasificó el payload de error como `Rechazada Producción`, mientras el portal indicaba `Pendiente de envío`. Esto demuestra que el estado guardado en AmayaSoft puede no reflejar el procesamiento posterior que realiza el proveedor.

El error apunta a configuración de certificado/clave privada del lado de Factiliza, no a las reglas del cálculo del total. La incidencia requiere confirmación/reprocesamiento por Factiliza y no debe resolverse creando otro correlativo ni reintentando a ciegas. Falta una acción de conciliación que consulte/sincronice el estado posterior del portal; hasta implementarla, el administrador debe revisar el proveedor y no depender únicamente de la etiqueta que muestra AmayaSoft.

10. Seguridad, consistencia y errores

Autenticación y autorización

- Las áreas privadas requieren usuario autenticado.
- Las operaciones administrativas se protegen con rol.
- Emitir y solicitar una anulación fiscal son POST protegidos con antiforgery y requieren rol Administrador.
- La emisión y anulación de Producción requieren reintroducir la contraseña actual.
- El detalle y los archivos se restringen al administrador o al usuario propietario de la venta.

Secretos y transporte

- Tokens y cadena de conexión se proporcionan como variables de entorno/configuración del hosting.
- No se incluyen valores secretos en el manual, `appsettings` publicado ni capturas.
- La aplicación redirige HTTP a HTTPS; en Production usa cookies Secure, HttpOnly y SameSite Lax.
- Las solicitudes Factiliza no siguen redirecciones automáticas.
- Los cuerpos de respuesta externa y archivos se limitan en tamaño; el parseo XML deshabilita DTD y resolución externa.

Consistencia de datos

- Ventas, caja, compras, gastos y ajustes críticos comparten bloqueo SQL de operaciones.
- Las ventas y compras usan transacciones para proteger múltiples escrituras relacionadas.
- Las operaciones se recalculan en servidor y los valores monetarios usan columnas decimales.
- Las pruebas concurrentes comprueban idempotencia de venta, stock y numeración fiscal.
- La operación en AmayaSoft y la respuesta fiscal son dos sistemas independientes: un fallo externo no borra una venta ya confirmada.

Gestión de resultados

Situación	Conducta que debe seguir el operador
Venta en AmayaSoft confirmada; factura pendiente	Revisar el detalle fiscal antes de enviar
Factiliza rechaza validación de datos antes de enviar	Corregir datos solo si consta que no salió solicitud HTTP
Timeout, respuesta truncada o estado pendiente	Consultar portal/soporte; no emitir de nuevo automáticamente
Aceptación con CDR código `0`	Registrar como aceptado; descargar archivos disponibles
Error de certificado de firma en producción	Escalar a Factiliza; no reutilizar número ni anular sin conocer el estado tributario

11. Alojamiento, despliegue y operación

Plataforma de alojamiento

La versión final de AmayaSoft está desplegada en MonsterASP.NET, en un sitio ASP.NET Core con base de datos Microsoft SQL Server. La dirección pública específica corresponde al sitio registrado en la cuenta de MonsterASP.NET y se administra desde su panel de control; no se incluye aquí porque no forma parte del código fuente. La aplicación requiere ejecutarse con el entorno `Production` y acceder a la base de datos del alojamiento mediante la cadena de conexión configurada para el sitio.

MonsterASP.NET ofrece publicación de aplicaciones ASP.NET mediante Visual Studio/Web Deploy, FTP/SFTP y otros métodos. El mecanismo concreto de la publicación vigente se gestiona desde la tarjeta Websites → sitio de AmayaSoft → Deploy del panel. El perfil de publicación contiene credenciales de despliegue y debe mantenerse privado; no se incluye en el PDF ni en el repositorio.

Configuración del sitio en MonsterASP.NET

En el panel de control de MonsterASP.NET:

1. En Websites, abrir el sitio asignado a AmayaSoft. La URL pública y el estado de la aplicación se muestran en la vista general del sitio.
2. En Databases, abrir la base MSSQL y obtener la cadena de conexión para acceso desde una aplicación alojada en MonsterASP.NET. Usar el valor del proveedor como secreto de configuración; no copiarlo al documento ni al control de versiones.
3. En Scripting → Environment variables, configurar `ASPNETCORE_ENVIRONMENT=Production` y las variables de aplicación necesarias.
4. Guardar los cambios y reiniciar la aplicación desde el panel para que ASP.NET Core reciba la configuración.
5. Mantener habilitado HTTPS para el dominio público.

MonsterASP.NET documenta que ASP.NET Core lee las variables de entorno automáticamente y que estas prevalecen sobre `appsettings.json`. Para las secciones anidadas de configuración de .NET se utiliza doble guion bajo (`__`).

Variables de operación

Los nombres de variables que requiere la integración son:

```
ASPNETCORE_ENVIRONMENT
ConnectionStrings__DefaultConnection
FactilizaSettings__FacturacionHabilitada
FactilizaSettings__Produccion__SeriesConfirmadas
FactilizaSettings__Produccion__EmpresaRuc
FactilizaSettings__Produccion__BoletaSerie
FactilizaSettings__Produccion__FacturaSerie
FactilizaSettings__Produccion__BoletaCorrelativoInicial
FactilizaSettings__Produccion__FacturaCorrelativoInicial
FactilizaSettings__Produccion__Token
FactilizaSettings__ConsultaToken
```

La cadena de conexión y el token son secretos: sus valores se configuran en el panel y no se guardan en el repositorio, en el PDF ni en archivos publicados. La facturación de Producción se habilita únicamente cuando la configuración fiscal está completa y el proveedor/administrador ha confirmado las series.

Publicación y actualización de esquema

La publicación Release se genera con:

```
dotnet publish MiniMarket.Web.csproj -c Release -o .\publicacion
```

La primera actualización de una base heredada es una operación separada y explícita, después de generar un respaldo y con permisos DDL temporales:

```
dotnet MiniMarket.Web.dll --apply-commercial-upgrade
```

El comando debe ejecutarse una sola vez desde un entorno autorizado para conectarse a la base de datos del sitio; aplica la actualización aditiva y termina sin iniciar el servidor web. Después, la aplicación debe operar con una identidad de base de datos de permisos mínimos. Producción no aplica actualizaciones de esquema automáticamente al iniciar.

Para futuras publicaciones, el flujo recomendado es: compilar y validar la versión, respaldar la base alojada, ejecutar cualquier actualización de esquema aprobada, publicar el artefacto en el sitio de MonsterASP.NET, revisar registros del sitio y confirmar que la aplicación responde por su URL HTTPS. Los datos de conexión, credenciales de Web Deploy, token de Factiliza y dominio específico se administran en el panel y no se exponen en esta documentación.

12. Pruebas y verificación

La suite de integración se ejecuta con:

```
dotnet run --project Tests/Comercial.IntegrationTests.csproj -c Release
```

La suite requiere una instancia de SQL Server de pruebas y permisos para crear bases temporales. Crea una base aislada con nombre aleatorio, ejecuta verificaciones y la elimina al finalizar. Los handlers HTTP de prueba simulan Factiliza; las pruebas automáticas no necesitan token fiscal ni emiten documentos.

Las garantías cubiertas incluyen validación de venta, autoridad de precios, cambios de stock transaccionales, concurrencia, cierres, bloqueo del sembrador fuera de Development, fechas de Perú, consultas del asistente, payload tributario, permisos, secuencias, respuestas externas inciertas y protección contra duplicación.

La comprobación de compilación utilizada para el proyecto fue:

```
dotnet build MiniMarket.Web.csproj -c Release
```

La verificación de paquetes vulnerables utilizada fue:

```
dotnet list MiniMarket.Web.csproj package --vulnerable --include-transitive
```

En la validación de código de esta versión, la suite finalizó correctamente, el build Release terminó sin errores ni advertencias y la auditoría NuGet no reportó paquetes vulnerables. Estos resultados describen pruebas de código, no homologación tributaria completa ni disponibilidad del certificado de Factiliza.

13. Limitaciones y estado conocido

1. Correo de comprobantes: no hay servicio SMTP ni botón de envío. El campo de correo de la ficha del cliente no se envía a Factiliza.
2. Sincronización fiscal: el sistema guarda la respuesta inicial, pero no consulta de fondo los reenvíos automáticos ni actualiza por sí solo un estado posterior del portal.
3. Certificado de Producción: se comunicó una falla de clave privada en la infraestructura de firma de Factiliza. La emisión requiere confirmación y resolución del proveedor.
4. Cobertura tributaria: la generación actual contempla operaciones gravadas modeladas por el servicio; otras modalidades fiscales requieren análisis e implementación propia.
5. Pronóstico: la recomendación de compra es una regla basada en promedio reciente y stock mínimo; debe revisarse por una persona responsable.
6. Anulación de venta y anulación fiscal: son acciones distintas. Restituir stock no equivale a que SUNAT anule el comprobante ni a reembolsar un pago.
7. Pruebas de extremo a extremo: la batería automatizada no navega el portal de Factiliza ni certifica que una emisión real sea aceptada por SUNAT.
8. Datos de demostración: el endpoint de generación manual solo está disponible en Development. La inicialización automática fuera de Development requiere habilitar explícitamente `StartupDatabaseTasks:Enabled` y no se ejecuta en Production. La creación inicial de la cuenta administrativa requiere `SeedData__InitialAdminPassword`.

14. Guía breve para la exposición

¿Qué arquitectura utiliza?

ASP.NET Core MVC. Controladores administran solicitudes, servicios contienen reglas de negocio, modelos representan entidades y vistas Razor presentan la interfaz. EF Core conecta las entidades con SQL Server.

¿Cómo se evita confiar en el navegador para el precio?

El servidor obtiene precio y stock desde SQL, valida el total esperado y recalcula antes de escribir. No utiliza como autoridad el precio enviado por JavaScript.

¿Cómo se evita que dos ventas consuman la última unidad?

Las operaciones toman un bloqueo SQL común y la fila del producto se lee con bloqueo durante la transacción. La prueba de concurrencia comprueba que solo una venta puede consumir stock insuficiente para la otra.

¿Qué diferencia existe entre venta y comprobante?

La venta registra el cobro interno, caja e inventario. El comprobante es una operación fiscal externa que solo inicia manualmente el administrador y requiere respuesta SUNAT/CDR.

¿Qué significa que Factiliza responda HTTP 200?

Solo significa que la API respondió a la solicitud. La aplicación exige también éxito SUNAT y CDR código `0` para considerar aceptado el documento.

¿Por qué no se debe reintentar un resultado incierto?

El proveedor podría haber recibido el documento aunque el navegador o servidor no guardara la respuesta. Reenviar sin reconciliar puede crear duplicados o conflictos de numeración.

¿El asistente es un modelo de IA externo?

No en la implementación actual. Interpreta patrones de texto y ejecuta consultas predefinidas sobre los datos, con límites por usuario/rol y sin generar SQL dinámico.

¿Por qué se guarda el costo histórico?

Porque el costo actual puede cambiar cuando se registra una nueva compra. El costo histórico permite que reportes futuros no reescriban el costo de ventas antiguas.

15. Glosario

Término	Significado en el proyecto
MVC	Separación entre modelo de datos, vista e interacción/controlador
EF Core	ORM de Microsoft usado para consultas y persistencia en SQL Server
Identity	Sistema de autenticación, usuarios, contraseñas y roles de ASP.NET Core
Idempotencia	Repetir la misma petición no duplica una venta ya confirmada
Transacción	Unidad de escritura que confirma todos sus cambios juntos o revierte ante error
Kardex	Historial de entradas, salidas y ajustes de inventario
CDR	Constancia de respuesta de SUNAT a un documento electrónico
Serie	Prefijo de identificación fiscal de un tipo de documento
Correlativo	Número secuencial asignado a un documento dentro de su serie
Ambiente QA	Ambiente de pruebas del proveedor; no equivale a emisión tributaria productiva
Resultado incierto	No se puede demostrar solo con el estado guardado en AmayaSoft si el proveedor aceptó o procesó la solicitud
Antiforgery	Protección de formularios POST contra solicitudes cruzadas no autorizadas

16. Mapa de archivos relevantes

Archivo	Función
`Program.cs`	Servicios, Identity, middleware, rutas y política de esquema al inicio
`Data/ApplicationDbContext.cs`	Tablas EF Core, relaciones e índices únicos
`Data/ActualizacionComercial.cs`	Actualización aditiva del esquema y bloqueo compartido
`Data/ActualizacionFiscal.cs`	Tabla e índices de comprobantes fiscales
`Controllers/VentasController.cs`	Punto de venta, historial, detalle fiscal, archivos y permisos de emisión
`Controllers/SeedDataController.cs`	Generación de datos de demostración restringida al entorno Development
`Services/OperacionVentaService.cs`	Validación, precio de autoridad SQL, transacción e idempotencia de venta
`Services/ComprobantePayload.cs`	Validaciones y construcción del JSON fiscal
`Services/FacturacionService.cs`	Envío, parsing de respuesta, CDR, archivos, numeración y anulaciones
`Services/FactilizaService.cs`	Cliente HTTP de consulta DNI/RUC y fábrica del servicio fiscal
`Services/IAAsistenteService.cs`	Interpretación y consultas de datos autorizadas para el asistente
`Models/`	Entidades persistidas y modelos para las vistas
`Views/`	Interfaz MVC por módulo
`Tests/Program.cs`	Ejecutable de pruebas de integración
`Tests/FacturacionQaTests.cs`	Pruebas fiscales simuladas y concurrencia
`README-ENTREGA.md`	Instrucciones de ejecución, despliegue y configuración de facturación

17. Referencias

Código fuente consultado

Los nombres de módulos, reglas, roles, tablas, validaciones y configuración se derivan de la copia editable de AmayaSoft incluida con este documento. En caso de diferencia, prevalece el comportamiento del código desplegado y su configuración efectiva.

Documentación externa de integración

- Factiliza, API SUNAT Facturación
- Factiliza, envío de factura y boleta
- Factiliza, documentación del API de consulta
- MonsterASP.NET, publicación con Visual Studio/Web Deploy
- MonsterASP.NET, variables de entorno
- MonsterASP.NET, conexión a bases MSSQL

Fin del manual. No se incorporaron filas de la base de datos, clientes reales, transacciones de ejemplo, capturas operativas ni credenciales.